K24: Systems Programming - 2nd Assignment, Spring Semester 2023

Submission Deadline: Sunday 11/6/2023, 23:59

Objective

The goal of this assignment is to familiarize yourselves with systems programming in a Unix environment, and specifically with the creation of threads and network communication. The assignment is divided into three parts.

Problem 1: Multi-threaded network server (50 points).

Description - Execution.

Write a multi-threaded network server in C/C++ called poller, which conducts a pre-election poll of political parties. You will run the program from the command line with the following arguments

```
prompt> poller [portnum] [numWorkerthreads] [bufferSize] [poll-log][poll-stats]
```

Specifically, the arguments are:

- **portnum:** the port number the server listens on
- **numWorkerthreads:** the number of worker threads it will create to conduct the poll. It must be > 0.
- **bufferSize:** the size of a buffer that will hold connections from clients waiting to be served. It must be > 0.
- **poll-log:** the name of a file where the names of users and their votes will be stored.
- **poll-stats:** the name of a file where the results will be stored (e.g., the number of users who have voted for each party).

All arguments are mandatory and must be given only in this specific order.

For example, if you run your program with the following arguments:

```
prompt> poller 5634 8 16 pollLog.txt pollStats.txt
```

then the server will listen on port 5634, create 8 worker threads, use a buffer that holds up to 16 connections waiting to be served, and store the users' votes in pollLog.txt and the poll results in the file pollStats.txt.

Implementation

The server you will implement will support

- the creation of a master thread
- the creation of the buffer for storing the connections
- updating the log file with the poll statistics

Master thread

In your implementation, you should have a master thread that starts by creating numWorkerThread worker threads. The master thread will accept connections from clients with the **accept** system call and will place the file descriptors corresponding to the connections into a buffer of specific size (defined by bufferSize). The master thread will NOT read from the connections it accepts. It will simply place the socket descriptor into the buffer whenever it accepts a connection and will continue accepting further connections.

Worker threads

The job of the worker threads is to read the requests from the socket descriptors and to serve the clients. A worker thread wakes up when there is at least one descriptor in the buffer (i.e., at least one client has connected to the server).
When it wakes up, the worker thread sends the message “SEND NAME PLEASE” and waits to read the reply (the name of a voter) from the client through the socket descriptor.
If the voter has already voted, it sends the reply “ALREADY VOTED” and terminates the connection.
Otherwise it sends the message “SEND VOTE PLEASE” and waits to read the reply (a string with the name of a political party).
It will print on one line, at the end of the poll-log file, the name of the voter and the party they voted for.
Finally it sends the message “VOTE for Party XYZ RECORDED” (where XYZ is the party the voter voted for) and terminates the connection.

Master-worker synchronization through the buffer.

The master thread and the worker threads have a producer-consumer relationship so in your implementation their accesses to the shared buffer must be synchronized.
Specifically:

- the master thread must block and wait when the buffer is full
- a worker thread must wait if the buffer is empty.

With this approach, if there are more worker threads than active connections, some of the worker threads will block, waiting for new connections to arrive at the server.

In this assignment, you must use condition variables in your implementation. **If your implementation does any busy-waiting, there will be a penalty (-20 points).**

Server - Storing Statistics

The server must keep some data about the poll:

- the names of the users who have participated in the poll so far together with their votes, and
- the number of users who have voted for each party.

In the implementation, the votes received by the server must be stored in the poll-log and poll-stats files as they are sent. Attention: if the server receives a termination signal, it must write the votes it has received up to that moment before terminating so that the result is correct. You can keep this data in any data structure that suits you.

As the threads serving the users must update the data kept by the server, **you must also implement the appropriate synchronization so that modifications to the data are made correctly**.

Regardless of the method chosen for updating the files, their final format will be as follows

poll-log file:

```
voter_name1 party_voted_for1
voter_name2 party_voted_for2
 .
 .
 .
```

The server will run until it receives a SIGINT (control-C) signal. When it receives SIGINT, it will write the poll results to the poll-stats file in the following format:

poll-stats file

```
party_name1   number_of_votes
party_name2   number_of_votes
party_name3   number_of_votes
 .
 .
 .
TOTAL total_number_of_votes
```

Problem 2: Batch client for testing purposes (30 points) Write a multi-threaded client in C/C++ called pollSwayer that takes three arguments:

```
prompt> pollSwayer [serverName] [portNum] [inputFile.txt]
```

where:

- **serverName:** the name of the server it will connect to
- **portNum:** the port number the server listens on
- **inputFile.txt:** a file that contains the names and votes of the voters

All arguments are mandatory and must be given only in this specific order.

For example, if you run your program with the following arguments:

```
prompt> pollSwayer linux01.di.uoa.gr 5634 inputFile.txt
```

pollSwayer will read `inputFile.txt` and for each line with a first name, last name and vote, it will create a thread that connects to the server `linux01.di.uoa.gr` on port 534 and sends it the vote following the protocol we described above. If you run this program from multiple different shells at the same time, it will help you find and fix synchronization bugs in your server.

Problem 3: Bash scripts (20 points). You will write a series of bash scripts which you will use for debugging your program. Of course, during the development of your program you can use a few small files for debugging. The first script `create_input.sh` works as follows (10 points):

```
./create_input.sh politicalParties.txt numLines
```

- `politicalParties.txt`: a file that contains the names of all the parties in the poll, one party per line.
- `numLines`: the number of lines the `inputFile` created by the script will contain.

The script does the following

1. Performs checks on the input arguments/numbers.
2. Creates a file `inputFile`.
3. Creates `numLines` lines in `inputFile` where each line contains a random first name and last name (random strings with random length from 3 to 12 characters) and the name of a party (the script will randomly choose a party from `politicalParties.txt`).

You may find the $RANDOM Bash function useful. (See, e.g. http://tldp.org/LDP/abs/html/randomvar.html)

The second script `tallyVotes.sh` works as follows (5 points):
`./tallyVotes.sh tallyResultsFile`

1. Checks whether an `inputFile` file exists in the current directory. If it does not exist or does not have the appropriate access permissions, it prints an error message and terminates.
2. Reads the `inputFile` file and counts the number of votes each party receives. If there is a duplicate voter, it counts only their first vote.
3. In a `tallyResultsFile` it places the results where each line contains the name of a party and the number of votes the party received.

The third script `processLogFile.sh` works as follows (5 points):
`./processLogFile.sh poll-log`

1. Checks whether a `poll-log` file exists in the current directory. If it does not exist or does not have the appropriate access permissions, it prints an error message and terminates.
2. Reads the `poll-log` file. For each party, it counts the number of votes that appear in `poll-log`. If there is a duplicate voter, it counts only their first vote.
3. In a `pollerResultsFile` it places the results where each line contains the name of a party and the number of votes the party received.

When you run the scripts `tallyVotes.sh` and `processLogFile.sh`, the output files produced (`tallyResultsFile` and `pollerResultsFile`) must come out exactly the same and fully match the results that `poller` produces in the poll-stats file when it terminates with a SIGINT (control-C) signal.

Procedural Matters

• Your program must be written in C (or C++) and must run on the Department's Linux workstations. Code that does not compile there is considered not to compile.

• For additional announcements, follow the course forum on piazza.com. The full address is https://piazza.com/uoa.gr/spring2023/k24/home. Following the forum on Piazza is mandatory.

• Your code must consist of at least two (and preferably more) different files. The use of separate compilation is mandatory and your code must have a Makefile.

• Make sure you follow good software engineering practices when implementing the assignment. The organization, readability and presence of comments in the code are part of your grade.

• Submission will be done through eclass.

• Your code must compile into the executables `poller` and `pollSwayer` **exactly** as the assignment specifies and have the same parameters exactly as the assignment specifies.

What to Submit

- All your work (source code, Makefile and README) in a tar.gz file named OnomaEponymoProject2.tar.gz. Be careful to submit only code, Makefile, README and not the binaries. Your assignment will be compiled from scratch before being graded.
- Any design choices you make must be described in a README (plain text file) that you will submit together with your code. The README needs to contain a brief and concise explanation of the choices you made in the design of your program. 1-2 pages of ASCII text are enough. Include the explanation and the instructions for compiling and running your program.
- The code you submit must be your own. The use of code not written by you or code written with the help of engines such as chatGPT is forbidden.
- It is a good idea to keep a backup .tar of your assignment exactly as it was submitted on some easily accessible machine (department server, github, cloud).
- Correctly submitting a correct tar.gz containing the code of your assignment and whatever files are needed is solely your responsibility. **Empty tar/tar.gz files or tar/tar.gz files that are faulty and cannot be extracted will not be graded.**

What will be graded

- The compliance of your code with the specifications of the assignment.
- The organization and readability (together with the presence of comments) of the code.
- The use of a Makefile and separate compilation.

Other Important Notes

- The assignments are individual.
- Anyone who submits / shows code that was not written by them gets a **zero** in the course.
- Although you are expected to discuss with friends and colleagues how you will attempt to solve the problem, copying code (of any form) is not allowed. Anyone found involved in code copying simply gets zero in the course. This applies to everyone involved regardless of who gave/took etc. We emphasize that you must take the appropriate measures so that your code is protected and not stored anywhere another user has access to (e.g., the excuse “I had put it in a github repo and someone probably took it from there” is not acceptable.)
- Programming assignments can be submitted at most 3 days late with a 5% penalty for each day of delay. Beyond these 3 days, assignments cannot be submitted.
- Your program must be written in C or C++. You may only use commands that are available on the Department's Linux machines. A program that may compile or run on your personal computer but not on the Department's Linux machines is considered not to compile or run respectively.